



CURSO BÁSICO DE PROGRAMACIÓN EN JAVA

MANEJO DE EXCEPCIONES

Tutor/es: Brandon Jiménez, Christopher Benavides,
Jhon Laverde, Dayana Jácome

1



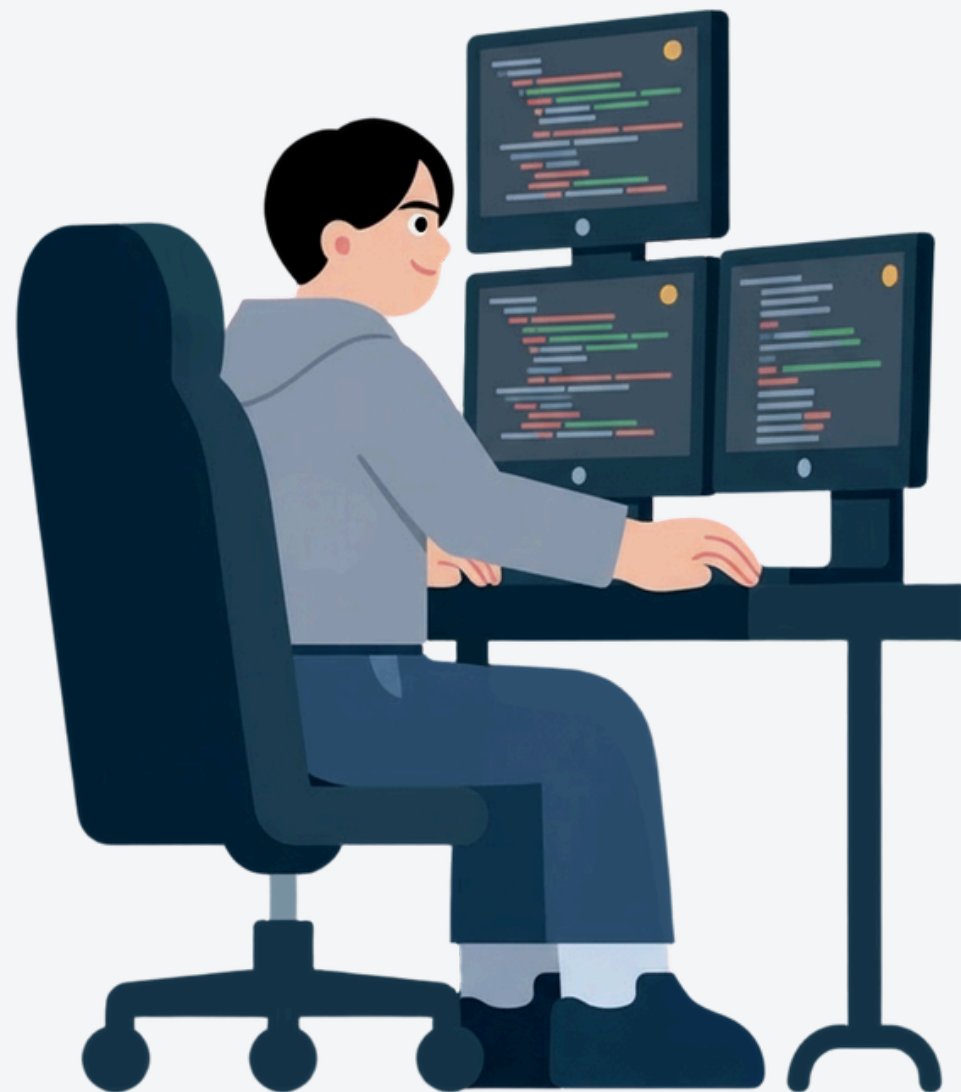
INDICE

- 01. Introducción
- 02. ¿Qué es una EXCEPCIÓN?
- 03. Tipos de ERRORES en programación
- 04. Uso de try - catch
- 05. InputMismatchException
- 06. Validación de datos del usuario





INTRODUCCIÓN



En programación, pueden ocurrir errores durante la ejecución de un programa, especialmente cuando el usuario ingresa datos incorrectos. Estos errores, llamados excepciones, pueden hacer que el programa se detenga si no se controlan. Por ello, se utilizan estructuras como try-catch y validaciones de datos



¿Qué es una EXCEPCIÓN?

Una excepción es un error que ocurre mientras el programa se está ejecutando (no antes).

Es decir:

- Tu programa compila bien
- Pero al ejecutarlo... algo falla

```
try {  
    // código que puede generar error  
} catch (TipoDeError e) {  
    // qué hacer si ocurre ese error  
}
```



Ejemplo:

Si pides un número y el usuario escribe letras → el programa se cae.



```
7 public static void main(String[] args) {  
8     // TODO Auto-generated method stub  
9  
10    Scanner sc = new Scanner(System.in);  
11  
12    System.out.println("Ingrese un número:");  
13    int numero = sc.nextInt(); // si ponen "hola" → E  
14  
15    System.out.println("Número: " + numero);  
16  
17 }  
18 }
```

Problems @ Javadoc Declaration Console X

terminated> Prueba1 [Java Application] C:\Users\USER\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp

Ingrese un número:

Dayana

Exception in thread "main" java.util.InputMismatchException
at java.base/java.util.Scanner.throwFor(Scanner.java:937)
at java.base/java.util.Scanner.next(Scanner.java:1602)
at java.base/java.util.Scanner.nextInt(Scanner.java:2267)
at java.base/java.util.Scanner.nextInt(Scanner.java:2221)
at progra1.Prueba1.main(Prueba1.java:13)

```
7 public static void main(String[] args) {  
8     // TODO Auto-generated method stub  
9  
10    try {  
11        Scanner sc = new Scanner(System.in);  
12        System.out.println("Ingrese un número:");  
13        int numero = sc.nextInt();  
14  
15        System.out.println("Número: " + numero);  
16    } catch (Exception e) {  
17        System.out.println("Error: Debes ingresar un número");  
18    }  
19 }
```

Problems @ Javadoc Declaration Console X

terminated> Prueba1 [Java Application] C:\Users\USER\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32

Ingrese un número:

Dayana

Error: Debes ingresar un número

TIPOS DE ERRORES EN PROGRAMACIÓN

ERROR SINTÁCTICO



Es cuando escribes mal el código y no cumple las reglas del lenguaje (Java).

El programa NO compila (ni siquiera corre).

```
System.out.println("Hola"
```



```
System.out.println("Hola");
```

```
int numero = "5";
```



```
int numero = 5;
```

```
if (5 > 3 {  
    System.out.println("Mayor");  
}
```



```
if (5 > 3) {  
    System.out.println("Mayor");  
}
```



TIPOS DE ERRORES EN PROGRAMACIÓN

ERROR LÓGICO



El programa funciona... pero el resultado es incorrecto
El error está en tu lógica (tu forma de pensar el problema)

```
int edad = 20;  
  
if (edad > 18) {  
    System.out.println("Es menor de edad");  
}
```



```
int edad = 20;  
  
if (edad < 18) {  
    System.out.println("Es menor de edad");  
}
```

```
double precio = 100;  
double total = precio * 0.10; // querías sumar impuesto
```



```
double precio = 100;  
double total = precio + (precio * 0.10);
```

TIPOS DE ERRORES EN PROGRAMACIÓN

ERROR EN TIEMPO DE EJECUCIÓN



Es un error que ocurre mientras el programa se está ejecutando.

Puede hacer que el programa se cierre si no se controla.

```
int x = 10 / 0;
```



```
int x = 10 / 2;
```

```
try {  
    int x = 10 / 0;  
} catch (Exception e) {  
    System.out.println("No se puede dividir para cero");  
}
```





USO DE TRY - CATCH

- Son estructuras en Java que sirven para manejar errores (excepciones) y evitar que el programa se cierre.

try → “intentar”

Es el bloque donde pones el código que puede fallar.

Significa:

“voy a intentar ejecutar esto”

catch → “atrapar”

Es el bloque que captura el error si ocurre.

Significa:

“si algo falla, hago esto en vez de que el programa se rompa”

7

INPUTMISMATCHEXCEPTION

Es una excepción específica cuando el usuario ingresa un tipo de dato incorrecto.

¿Cuándo ocurre?

Principalmente con Scanner cuando usas:

```
sc.nextInt();  
sc.nextDouble();
```



¿Por qué se usa?

Se usa para:

- Detectar errores específicos del usuario
- Mostrar mensajes claros
- Evitar que el programa se cierre



VALIDACIÓN DE DATOS DEL USUARIO

No basta con usar try-catch, también debes verificar que los datos tengan sentido.

¿POR QUÉ ES TAN IMPORTANTE?

Porque el usuario puede ingresar:

- Números negativos

- Valores absurdos (edad = 500)

- Datos vacíos

- Valores que no tienen sentido

Y el programa igual funcionaría... pero mal

```
try {  
    System.out.println("Ingresa edad:");  
    int edad = sc.nextInt();  
} catch (Exception e) {  
    System.out.println("Error");  
}
```



CONCLUSIONES

- 1** .Las excepciones permiten manejar errores que ocurren durante la ejecución del programa, evitando que este se detenga de forma inesperada
- 2** Es importante diferenciar los errores:
 - Sintácticos: se detectan al escribir el código y no permiten compilar.
 - Lógicos: el programa funciona, pero da resultados incorrectos.
 - En tiempo de ejecución: ocurren al ejecutar el programa y pueden detenerlo si no se controlan.
- 3** El uso de try-catch es fundamental para controlar errores, ya que permite ejecutar código de forma segura y definir qué hacer en caso de fallos
- 4** La validación de datos es esencial para asegurar que la información ingresada sea correcta y tenga sentido

11



TAREA

Desarrolle un programa en Java llamado **“Tienda”** que cumpla con lo siguiente:

1. Ingreso de datos

2. Validaciones obligatorias
(nombre, cantidad, precio)

3. Manejo de errores

4. Procesamiento

- $\text{total} = \text{cantidad} * \text{precio}$
- Aplicar descuento del 10% si el total es mayor a \$100
- 5. Repetición → Preguntar al usuario si desea realizar otra compra (s/n)

12

EJEMPLO

```
public class Tienda {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        String nombre;  
        int cantidad = 0;  
        double precio = 0;  
        double total;  
        char opcion;
```

```
        // =====  
        // REPETIR  
        // =====  
        System.out.println("\n¿Desea ingresar otra compra? (s/n):");  
        sc.nextLine(); // limpiar  
        opcion = sc.nextLine().charAt(0);  
  
        } while (opcion == 's' || opcion == 'S');  
  
        sc.close();  
        System.out.println("Programa finalizado.");  
    }  
}
```

```
Ingrese el nombre del cliente:  
123  
Error: El nombre no debe contener números.  
Ingrese el nombre del cliente:  
Dayana  
Ingrese la cantidad de productos:  
abc  
Error: Debe ingresar un número entero.  
Ingrese la cantidad de productos:  
2  
Ingrese el precio del producto:  
zzz  
Error: Debe ingresar un número válido.  
Ingrese el precio del producto:  
10  
  
--- FACTURA ---  
Cliente: Dayana  
Cantidad: 2  
Precio: $10.0  
Total: $20.0  
  
¿Desea ingresar otra compra? (s/n):  
no  
Programa finalizado.
```

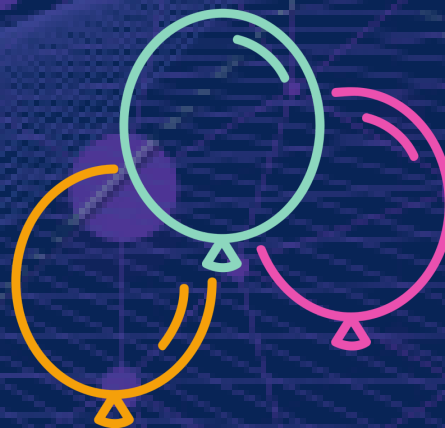



GRACIAS



Confía en tu
esfuerzo, estás
abriendo puertas
que aún no ves.

GLOWMEESS.COM





GRACIAS



Friday

